



Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 8

Image Classification

ConvNets from first principles, then the three strategies that make small image datasets tractable — from 80% to 98.6% on the same 2,000 pictures.

Duration 3 hours (2 sessions)

Instructor Rahman Indra Kesuma, S.Kom., M.Cs. · Prof. Bambang Riyanto Trilaksono

Source Chollet & Watson, Deep Learning with Python 3e — chapter 8

Session Objectives

By the end of this session, participants can:

- 1 Explain why a convolution layer beats a dense layer on images: **translation invariance** and **spatial hierarchies**.
- 2 Read a ConvNet summary and predict how **feature-map shape and depth** change layer by layer.
- 3 Say what **max pooling** is for, and what breaks in a ConvNet without it.
- 4 Build an image pipeline with `image_dataset_from_directory` and `tf.data`, including prefetching.
- 5 Apply **data augmentation** and explain what it can and cannot fix.
- 6 Use a pretrained model two ways — **fast feature extraction** and **end-to-end with augmentation** — and know the trade-off.
- 7 **Fine-tune** a pretrained base in the right order, at the right learning rate, and know which layers to leave frozen.

BOOK

[Chapter 8 — full text](#)

NOTEBOOK

[01_convnet_on_mnist.ipynb](#)

NOTEBOOK

[02_small_dataset_from_scratch.ipynb](#)

NOTEBOOK

[03_data_augmentation.ipynb](#)

NOTEBOOK

[04_feature_extraction_and_finetuning.ipynb](#)

NOTEBOOK

[All chapter 8 notebooks](#)

REPO

[Author's official notebook](#)

One problem, three strategies, four numbers

≈80%

small ConvNet from scratch, no regularisation

≈84%

the same, with data augmentation

98.1%

feature extraction from a pre-trained model

98.6%

fine-tuning that pretrained model

All

four use the **same 2,000 training images**. The lesson of the chapter is in the gap between the second number and the third — **reusing learned features is worth more than any amount of tuning your own small model**

01

Introduction to ConvNets

Why a convolution beats a dense layer on images.

A basic ConvNet is a stack of two layer types

listing 8.1 – a small ConvNet

PYTHON

```
import keras
from keras import layers

inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(inputs)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(10, activation="softmax")(x)
model = keras.Model(inputs=inputs, outputs=outputs)
```

A ConvNet takes tensors of shape `(image_height, image_width, image_channels)`, not counting the batch dimension.

What the summary tells you

OUTPUT

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 28, 28, 1)	0
conv2d (Conv2D)	(None, 26, 26, 64)	640
max_pooling2d (MaxPooling2D)	(None, 13, 13, 64)	0
conv2d_1 (Conv2D)	(None, 11, 11, 128)	73,856
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 128)	0
conv2d_2 (Conv2D)	(None, 3, 3, 256)	295,168
global_average_pooling2d	(None, 256)	0
dense (Dense)	(None, 10)	2,570

Total params: 372,234 (1.42 MB)

- ♦ **Width and height shrink** as you go deeper: $28 \rightarrow 26 \rightarrow 13 \rightarrow 11 \rightarrow 5 \rightarrow 3$.
- ♦ **Depth grows**: $1 \rightarrow 64 \rightarrow 128 \rightarrow 256$, controlled by the first argument to `Conv2D`.

The result, against the dense network from chapter 2

listing 8.3 – training it

PYTHON

```
train_images = train_images.reshape((60000, 28, 28, 1)).astype("float32") / 255
test_images = test_images.reshape((10000, 28, 28, 1)).astype("float32") / 255

model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_images, train_labels, epochs=5, batch_size=64)

test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Test accuracy: {test_acc:.3f}")
```

OUTPUT

Test accuracy: 0.991

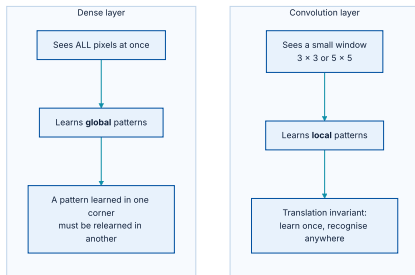
97.8% → 99.1%

dense network vs ConvNet

≈ 60%

relative reduction in error rate

The fundamental difference: global versus local

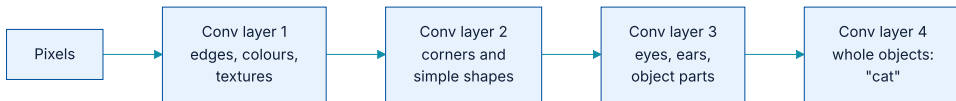


The one architectural change that produced that 60% error reduction.

Property 1 — translation invariance

This makes ConvNets **data efficient** on images, because the visual world is fundamentally translation invariant. They **need fewer training samples** to learn representations that generalise.

Property 2 — spatial hierarchies



Elementary lines and textures combine into eyes and ears, which combine into high-level concepts.

This works because **the visual world is itself spatially hierarchical**. The architecture matches the structure of the data — which is exactly what chapter 5 called a good **architecture prior**

Feature maps, filters, and response maps

- ♦ The output's depth is a **parameter of the layer**, and its channels no longer stand for colours — they stand for **filters**.
- ♦ A filter encodes a specific aspect of the input; at a high level, one filter might encode *"presence of a face"*.
- ♦ In the MNIST example the first layer turns $(28, 28, 1)$ into $(26, 26, 64)$ — it computes **64 filters** over its input.
- ♦ Each of those 64 channels is a 26×26 grid: a **response map** saying **how strongly that filter's pattern appears at each location**.

How the convolution actually runs

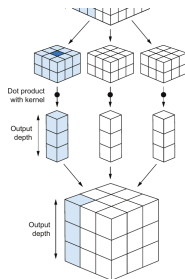
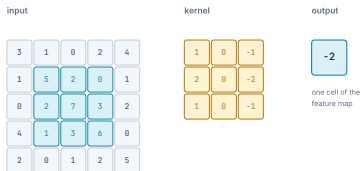


Figure 8.4 — a window slides over the input, each 3D patch is turned into a vector by the same learned kernel, and the vectors are reassembled into an output map. — Chollet & Watson, *Deep Learning with Python*, 3rd ed. (Manning)

Three things to take from the picture. The window moves over **every** position. The **same** kernel is used at each one — that is weight sharing. And the outputs are reassembled in the positions they came from, which is why the result is still an image and not a list. **The next slide computes one of those positions.**

One position of that window, computed



$$5 \times 1 + 0 \times -1 + 2 \times 2 + 3 \times -2 + 1 \times 1 + 6 \times -1 = -2$$

Nine multiplications and an addition. That is the whole operation; everything else is where you slide it to next.

A 3×3 kernel over one 3×3 patch. This kernel is a Sobel filter, which responds to vertical edges.

Reading the arithmetic: three things it settles

- 1 **A convolution is multiply - and - add, and nothing else.** Nine products and a sum give one output cell. Slide the window to the next position and do it again — that is the whole operation.
- 2 **The kernel is what gets learned.** Its nine numbers are `1, 0, -1 / 2, 0, -2 / 1, 0, -1` here because a person chose a Sobel filter. In a ConvNet **those nine numbers are parameters**, found by gradient descent, and nobody chooses what they end up detecting.
- 3 **The same nine numbers are reused at every position.** That is weight sharing: it is why a ConvNet has so few parameters compared with a `Dense` layer over the same image, and why a pattern learned in one corner is recognised in the other.

Why this kernel finds vertical edges: the left column is positive and the right column is negative, so the sum is large wherever **the left side is brighter than the right**, and near zero on flat ground. Run it on a patch where every cell is equal and the answer is exactly 0.

The two parameters that define a convolution

PARAMETER	TYPICAL VALUE	WHAT IT CONTROLS
Patch size	3×3 or 5×5	How large a neighbourhood each output value can see.
Output depth	32 to 512	How many filters are computed — how many different patterns the layer can look for.

how Keras spells it

PYTHON

```
layers.Conv2D(output_depth, (window_height, window_width))

# the same kernel is reused for every patch, which is why the layer is
# translation invariant and why it has so few parameters
```

Because the kernel is shared across all positions, **parameter count depends on the window and depth, not on the image size**

Max pooling: downsample, aggressively

	CONVOLUTION	MAX POOLING
Transformation	A learned linear transform (the kernel).	A hardcoded max operation.
Typical window	3×3 , stride 1	2×2 , stride 2
Effect on size	Shrinks slightly (no padding).	Halves each spatial dimension.

What happens without it

listing 8.5 – an incorrectly structured ConvNet

PYTHON

```
inputs = keras.Input(shape=(28, 28, 1))
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(inputs)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(10, activation="softmax")(x)
model_no_max_pool = keras.Model(inputs=inputs, outputs=outputs)
```

It is not conducive to learning a spatial hierarchy. The 3×3 windows in the third layer still only see information from **7×7 windows of the original image** — far too small a view to assemble a whole digit.

02

Training from scratch on a small dataset

2,000 pictures of cats and dogs. A situation you will actually meet.

The setup, and why it is realistic

2,000

training images

2,000

test images

1,000

validation images

Drawn from the original Kaggle Dogs vs. Cats dataset, which was released as a competition in late 2013 — **before ConvNets were mainstream**

Why deep learning still applies with little data

ConvNets are data efficient

Because they learn **local, translation-invariant** features, they give reasonable results on small image datasets without any custom feature engineering.

Models are repurposable

A classifier trained on a large dataset can be reused on a significantly different problem with minor changes. **Feature reuse is one of deep learning's greatest strengths.**

What counts as *enough* is relative — to the size and depth of the model. A few tens of samples will not do; a few hundred can suffice if the model is small, well regularised, and the task is simple.

A bigger ConvNet for bigger images

listing 8.7 — dogs vs cats, from scratch

PYTHON

```
inputs = keras.Input(shape=(180, 180, 3))
x = layers.Rescaling(1.0 / 255)(inputs)      # 0..255 -> 0..1, inside the model
x = layers.Conv2D(filters=32, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=64, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=128, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=256, kernel_size=3, activation="relu")(x)
x = layers.MaxPooling2D(pool_size=2)(x)
x = layers.Conv2D(filters=512, kernel_size=3, activation="relu")(x)
x = layers.GlobalAveragePooling2D()(x)
outputs = layers.Dense(1, activation="sigmoid")(x)    # binary -> one sigmoid unit
model = keras.Model(inputs=inputs, outputs=outputs)
```

Putting `Rescaling` in the model means the deployed artefact carries its own preprocessing — one fewer thing to get wrong in production.

The pattern you will see in almost every ConvNet



Depth rises from 32 to 512 while the spatial size falls from 180×180 to 7×7 .

Feature - map depth increases while feature - map size decreases. Information moves out of *where* and into *what* — and that is the whole shape of a classification ConvNet

From JPEG files to batched tensors

- 1 Read the picture files.
- 2 Decode the JPEG content into RGB grids of pixels.
- 3 Convert those into floating-point tensors.
- 4 Resize them to a shared size — here 180×180 .
- 5 Pack them into batches.

`image_dataset_from_directory` does all five

listing 8.9 – building the pipeline

PYTHON

```
from keras.utils import image_dataset_from_directory

batch_size = 64
image_size = (180, 180)

train_dataset = image_dataset_from_directory(
    new_base_dir / "train", image_size=image_size, batch_size=batch_size)
validation_dataset = image_dataset_from_directory(
    new_base_dir / "validation", image_size=image_size, batch_size=batch_size)
test_dataset = image_dataset_from_directory(
    new_base_dir / "test", image_size=image_size, batch_size=batch_size)
```

The directory layout **is** the label set: one subfolder per class, and the folder names become the classes.

What a `tf.data.Dataset` buys you

- ♦ It works with **any backend** — JAX and PyTorch included — not only TensorFlow.
- ♦ It is an **iterator**: usable in a `for` loop, and passable straight to `fit()`.
- ♦ It **parallelises preprocessing across CPU cores**.
- ♦ It does **asynchronous prefetching**: preparing the next batch while the model is still working on the previous one, so **execution never stalls**.

Those last two are the difference between a GPU that is busy and a GPU that spends most of its time waiting for JPEGs to decode.

03

Data augmentation

Making more training data out of the training data you have.

The idea, and where the layers go

WHERE YOU PUT THE LAYERS	RUNS ON	TRADE-OFF
Inside the model, before <code>Rescaling</code>	GPU	Sometimes faster, but the augmentation ships with the model.
In the data pipeline, via <code>map()</code>	CPU, in parallel	Usually the better option — and the one the book takes.

Three transformations, and prefetching

listing 8.18 – the augmentation stage

PYTHON

```
data_augmentation_layers = [  
    layers.RandomFlip("horizontal"),  
    layers.RandomRotation(0.1),  
    layers.RandomZoom(0.2),  
]  
  
def data_augmentation(images, targets):  
    for layer in data_augmentation_layers:  
        images = layer(images)  
    return images, targets  
  
augmented_train_dataset = train_dataset.map(data_augmentation, num_parallel_calls=8)  
augmented_train_dataset = augmented_train_dataset.prefetch(tf.data.AUTOTUNE)
```

- ♦ `RandomFlip("horizontal")` — flips a random **50%** of images.
- ♦ `RandomRotation(0.1)` — rotates by a random value in **`[-10%, +10%]`** of a full circle, i.e. **± 36 degrees**.
- ♦ `RandomZoom(0.2)` — zooms in or out by a random factor in **`[-20%, +20%]`**.

What augmentation cannot do

The model will never see the same input twice — but the inputs it does see are **still heavily intercorrelated**, because they come from a small number of originals. **You cannot produce new information; you can only remix existing information.**

≈ **80%**

from scratch, no augmentation

≈ **84%**

from scratch, with augmentation

Four points for a change that costs nothing but CPU time — worth having, and clearly not enough on its own.

04

Using a pretrained model

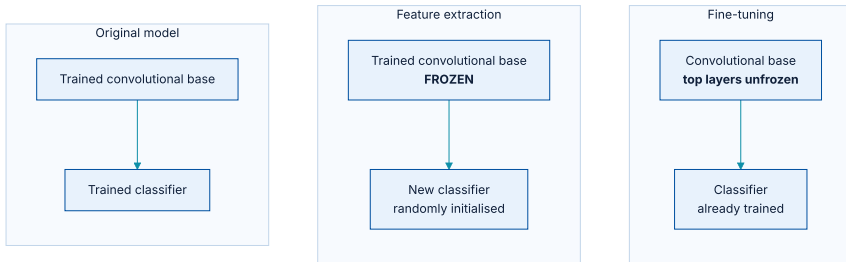
Where the remaining fourteen points come from.

Why features transfer between problems

The book uses **Xception** trained on ImageNet: 1.4 million labelled images across 1,000 classes. ImageNet contains many cat and dog breeds, so it should transfer well here.

Such portability across problems is **a key advantage of deep learning over older shallow methods**, and it is what makes it effective on small-data problems at all.

Two ways to reuse it



Feature extraction swaps the classifier; fine-tuning then also adjusts the top of the base.

Why only the convolutional base is reused

- ♦ The base produces **presence maps of generic concepts** — useful regardless of the vision problem.
- ♦ The classifier's representations are **specific to the classes it was trained on**.
- ♦ Worse, dense layers **discard spatial information**: they no longer know *where* anything is. For problems where location matters they are **largely useless**.

How much of the base to reuse

So if your new dataset differs a lot from the original, you may be better off using **only the first few layers** for feature extraction rather than the entire base.

Loading a pretrained backbone

listing 8.23–8.24 – backbone and preprocessing

PYTHON

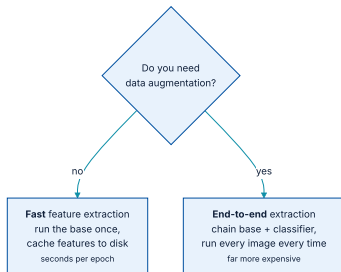
```
import keras_hub

conv_base = keras_hub.models.Backbone.from_preset("xception_41_imagenet")

preprocessor = keras_hub.layers.ImageConverter.from_preset(
    "xception_41_imagenet",
    image_size=(180, 180),
)
```

- ♦ **backbone** is KerasHub's word for the feature extractor without its classification head.
- ♦ The **41** is a naming convention: 41 trainable layers. Pretrained ConvNets are conventionally named by depth.
- ♦ `ImageConverter` matters more than it looks: every pretrained ConvNet expects a particular input range, and feeding the wrong one forces the model to **relearn how to see**.

Fast, or augmentable — pick one



The base is by far the most expensive part of the pipeline, so running it once versus every epoch is the whole trade-off.

Fast route — cache the features once

listing 8.25 — extracting features

PYTHON

```
def get_features_and_labels(dataset):
    all_features, all_labels = [], []
    for images, labels in dataset:
        preprocessed_images = preprocessor(images)
        features = conv_base.predict(preprocessed_images, verbose=0)
        all_features.append(features)
        all_labels.append(labels)
    return np.concatenate(all_features), np.concatenate(all_labels)

train_features, train_labels = get_features_and_labels(train_dataset)
val_features, val_labels = get_features_and_labels(validation_dataset)
test_features, test_labels = get_features_and_labels(test_dataset)

print(train_features.shape)
```

OUTPUT

```
(2000, 6, 6, 2048)
```

...then train a tiny classifier on the cache

listing 8.26 – the dense classifier

PYTHON

```
inputs = keras.Input(shape=(6, 6, 2048))
x = layers.GlobalAveragePooling2D()(inputs)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.25)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])

history = model.fit(train_features, train_labels, epochs=10,
                    validation_data=(val_features, val_labels),
                    callbacks=callbacks)
```

OUTPUT

```
validation accuracy: slightly over 98%
test accuracy       : 0.981
(an epoch takes under 1 second, even on CPU)
```

Read that result carefully

It is a **slightly unfair comparison**: ImageNet contains many dog and cat instances, so the pretrained model already holds **the exact knowledge this task needs**. That will not always be true.

The curves also show **overfitting almost from the start**, despite a fairly large dropout rate — because this route cannot use augmentation, which is essential on small image datasets.

Slow route — freeze the base and chain it

listing 8.28–8.29 — freezing

PYTHON

```
conv_base = keras_hub.models.Backbone.from_preset(
    "xception_41_imagenet",
    trainable=False,
)

conv_base.trainable = True
print(len(conv_base.trainable_weights))    # before freezing
conv_base.trainable = False
print(len(conv_base.trainable_weights))    # after freezing
```

OUTPUT

```
26
0
```

Setting `trainable = False` **empties the list of trainable weights** — which is exactly how you verify that a freeze took effect.

The chained model, and a recompile trap

the end-to-end model

PYTHON

```
inputs = keras.Input(shape=(180, 180, 3))
x = preprocessor(inputs)
x = conv_base(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(256)(x)
x = layers.Dropout(0.25)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
model = keras.Model(inputs, outputs)
model.compile(loss="binary_crossentropy", optimizer="adam", metrics=["accuracy"])
```

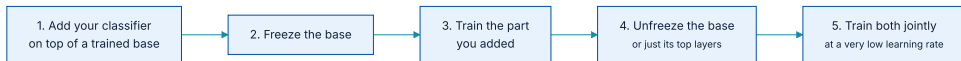
Only the two `Dense` layers train — four weight tensors in total. And a trap worth memorising: **if you change trainability after compiling, you must recompile, or the change is silently ignored**

05

Fine-tuning

Unfreezing the top of the base — carefully, and last.

The five steps, and why the order is fixed



Steps 1–3 are exactly the feature-extraction workflow. Fine-tuning is what you do afterwards.

The base can only be fine-tuned **once the classifier on top is already trained**. Otherwise the error signal propagating back is too large, and **the representations being fine-tuned are destroyed**

Partial fine-tuning, and why the top layers

- ♦ Earlier layers encode **generic, reusable** features; higher ones encode **specialised** features.
- ♦ It is the specialised ones that need repurposing, so there are **fast-decreasing returns** to fine-tuning lower layers.
- ♦ The base has **15 million parameters** — training all of them on 2,000 images risks overfitting.

One layer type to leave alone: **do not unfreeze** `BatchNormalization` layers. Chapter 9 explains why.

unfreezing the top four

PYTHON

```
conv_base.trainable = True
for layer in conv_base.layers[::-4]:
    layer.trainable = False
```

Fine-tune at a very low learning rate

listing 8.31 – fine-tuning

PYTHON

```
model.compile(
    loss="binary_crossentropy",
    optimizer=keras.optimizers.Adam(learning_rate=1e-5),    # note: 1e-5
    metrics=["accuracy"],
)

callbacks = [keras.callbacks.ModelCheckpoint(
    filepath="fine_tuning.keras", save_best_only=True, monitor="val_loss")]

history = model.fit(augmented_train_dataset, epochs=30,
                    validation_data=validation_dataset, callbacks=callbacks)
```

Updates that are too large **harm the representations you are trying to keep**. This is one of the few places in the book where the learning rate is prescribed rather than searched for.

The final number, in context

evaluating the fine-tuned model

PYTHON

```
model = keras.models.load_model("fine_tuning.keras")
test_loss, test_acc = model.evaluate(test_dataset)
print(f"Test accuracy: {test_acc:.3f}")
```

OUTPUT

Test accuracy: 0.986

In the original Kaggle competition this **would have been among the top results** — though not a fair comparison, since the pretrained features already carried prior knowledge about cats and dogs that competitors could not use.

The fairer point: this was reached using **about 10% of the training data** available in that competition. **There is a huge difference between training on 20,000 samples and on 2,000.**

What has to survive this chapter

- 1 **Convolutions learn local patterns**, which buys translation invariance and spatial hierarchies — and makes ConvNets data efficient on images.
- 2 **Depth rises as spatial size falls.** That shape is nearly universal in classification ConvNets.
- 3 **Max pooling is not optional:** without it, deep layers still only see a tiny window of the original image.
- 4 `image_dataset_from_directory` + `tf.data` turns a folder of JPEGs into a parallel, prefetching pipeline.
- 5 **Augmentation remixes information; it cannot create it.** Worth about four points here.

...and the part that changes how you plan a project

Reuse beats tuning

Feature extraction from a pretrained base took the same 2,000 images from **84% to 98.1%** — more than anything you could do to a small model of your own.

Fine - tune last, and gently

Only after the new classifier is trained, at a **very low learning rate**, and with `BatchNormalization` left frozen.

Which is why the first question on a new vision problem is not *what architecture?* but *whose features can I start from?*

NOTEBOOK

[04_feature_extraction_and_finetuning.ipynb](#)

NEXT

[Chapter 9 — ConvNet architecture patterns](#)